JSON REFINER ADVANCED EDITION

Project Report

Submitted to **Manonmaniam Sundaranar** University in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

NAAN MUDHALVAN COURSE
(Agentic AI Application Developer)

Submitted By

S. JARIN FATHIMA	**23081021802112014**
F. NASHMIA	**23081021802112022**
S. SIDDI SASMIN FATHIMA	**23081021802112030**
S. UMAIRA SABREEN	**23081021802112031**

Under the guidance of

Mrs. Dr. A. AAISHA NAZLEEM ,MCA.,M.Phil.,Ph.D

Assistant Professor,

DEPARTMENT OF COMPUTER SCIENCE

ANNAI HAJIRA WOMEN'S COLLEGE

(AFFILIATED TO MANONMANIAM SUNDARANAR UNIVERSITY)

TIRUNELVELI - 627005

MARCH– 2026

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
1.	**ABSTRACT**	01
2.	**INTRODUCTION**	03
3.	**SYSTEM REQUIREMENTS** 3.1 HARDWARE REQUIREMENTS 3.2 SOFTWARE REQUIREMENTS 3.3 CORE TECHNOLOGIES	06
4.	**SYSTEM DESIGN** 4.1 TECHNICAL ARCHITECTURE 4.2 PROJECT STRUCTURE	08
5.	**PROJECT WORKFLOW**	11
6.	**ALGORITHM**	13
7.	**IMPLEMENTATION** 7.1 CODING 7.2 OUTPUT	16
8.	**CONCLUSION**	37

ABSTRACT

1.ABSTRACT

The increasing dependency on data-driven applications in fields such as software development, data analytics, and API integration has made JSON (JavaScript Object Notation) the most critical format for information exchange. However, managing inconsistent data types, unformatted structures, and non-standard naming conventions remains a significant challenge for developers and data scientists. Writing and validating complex JSON manually is time-consuming and prone to syntax errors. To simplify this process and ensure high data integrity, the **JSON Refiner Advanced Edition** is developed as an intelligent automated processing tool. The main objective of this system is to help users transform raw, unstructured key-value data into professional, production-ready JSON formatted code without requiring deep manual effort.In this system, the user provides basic inputs such as raw key-value pairs or unrefined JSON strings. Based on these inputs, the application automatically performs **Automatic Type Inference**, identifying strings, numbers, booleans, and null values with high precision. The system includes several advanced features to improve usability: users can standardize their data using four different naming conventions—**snake_case, camelCase, kebab-case, and PascalCase**. Furthermore, the project supports complex operations like **Flattening and Unflattening** of nested JSON structures, allowing users to switch between dot-notation and hierarchical views easily.Another standout feature of this project is the **Schema Validation and History Tracking**, which ensures that the generated JSON adheres to structural standards while providing a log of all modifications made. The application also includes a **Merge JSON** feature, enabling users to combine multiple data sources into a single, cohesive structure. The project is implemented using **Python** for the backend processing engine and **Gradio** for a modern, tab-based interactive user interface. Overall, the JSON Refiner Advanced Edition provides a simple, efficient, and professional solution for data refinement, making complex JSON manipulation accessible to both technical and non-technical users.

INTRODUCTION

2.INTRODUCTION

In the modern digital world, data has become the backbone of communication, business intelligence, and software integration. Among various data formats, JSON (JavaScript Object Notation) plays a critical role as the standard for exchanging information between servers and web applications. Almost every modern API, mobile app, and cloud service relies on JSON to structure and transmit data. However, as data grows in complexity, maintaining its quality and consistency becomes a significant challenge. Developers and data analysts often deal with "messy" data—inconsistent naming conventions, incorrect data types (like numbers stored as strings), and deeply nested structures that are difficult to read or process.

To overcome these difficulties, the JSON Refiner Advanced Edition is developed as an intelligent, automated data processing platform. The main aim of this system is to simplify the complex task of JSON transformation and validation by providing a suite of automated tools. Instead of manually editing thousands of lines of code or writing custom scripts for every data set, users can provide raw key-value inputs or unrefined JSON strings. The system then processes this input using advanced algorithms to automatically infer correct data types, standardize naming conventions, and generate production-ready JSON code. This reduces manual effort and ensures that the data is structurally sound and ready for use in professional environments.

One of the most important features of this system is its Intelligent Type Inference and Case Conversion engine. The application can automatically detect whether a piece of data is a string, a number, a boolean, or a null value, correcting common formatting errors instantly. Furthermore, it supports four major industry-standard naming conventions: snake_case, camelCase, kebab-case, and PascalCase. This allows teams to maintain a consistent coding style across different projects and programming languages, which is essential for large-scale software development.

Another useful component of the project is the Advanced Structural Transformation feature. The system provides tools for "flattening" and "unflattening" JSON. Flattening converts complex, nested hierarchies into a simple dot-notation format, which is highly useful for data analysis in tools like Excel or SQL. Conversely, unflattening allows users to rebuild complex nested objects from simple lists. This flexibility helps beginners and professionals alike understand and manipulate how modern data structures are organized.

The system also includes a JSON Schema Validation and Merge feature. The validation component analyzes the generated JSON against predefined structural rules to ensure it meets modern standards, checking for required fields and proper data hierarchy. The Merge feature allows users to combine multiple JSON objects into one, solving the common problem of data fragmentation. By providing real-time feedback and a detailed history log of all changes, the system ensures transparency and accuracy throughout the refinement process.

The JSON Refiner Advanced Edition is implemented using Python for the powerful backend processing engine, while the Gradio framework is used to provide a modern, interactive web interface. By combining automation, structural validation, and a user-friendly design, the project provides an efficient solution for data refinement and helps users master the complexities of JSON data management with ease.

SYSTEM REQUIREMENTS

3.SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

1. **Processor:** Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz (or any modern x64-based processor).
2. **Installed RAM:** 8.00 GB (Minimum 4GB required for browser and local processing).
3. **System Type:** 64-bit Operating System, x64-based processor.
4. **Storage:** 500 MB free disk space (for Python environments and temporary data).
5. **Internet Connection:** Required for Google Colab and Gradio interface hosting.

3.2 SOFTWARE REQUIREMENTS

Operating System: Windows 10 / 11 or Linux/MacOS.

Programming Language: Python 3.8 or higher.

Libraries & Frameworks: * **Gradio:** For building the interactive web interface.

- **JSON5:** For flexible JSON parsing.
- **JSONSchema:** For structural data validation.
- **Pydantic:** For data models and type safety.

Development Tool: Google Colab (Cloud-based environment) / VS Code.

Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge (Latest versions).

5.3 CORE TECHNOLOGIES

- **Python Type System:** Used for automatic type inference (detecting strings, numbers, booleans).
- **Regex (Regular Expressions):** Used for advanced case conversion (camelCase, snake_case, etc.).
- **Gradio Tabs:** Used to organize the 7 functional modules (Core Refining, Schema, Flatten, Merge, etc.).
- **JSON Schema (Draft 7):** Used for production-grade structural validation.

SYSTEM DESIGN

4.SYSTEM DESIGN

The system design focuses on a modular architecture that separates the core data processing logic from the user interface. This ensures that the application is scalable, easy to maintain, and capable of handling complex JSON transformations efficiently.

7.1 Technical Architecture

The architecture follows a Model-View-Controller (MVC) inspired approach:

The Model (Data Engine): Handled by the JSONRefinerCore class, which manages the internal state of the JSON data, type inference logic, and the conversion history.

The Controller (Processing Layer): This layer contains the algorithms for Case Conversion (Regex-based), Flattening/Unflattening (Recursive logic), and Schema Validation (jsonschema integration).

The View (Interface Layer): Built using the Gradio framework, providing a web-based dashboard where users interact with the processing layers through a multi-tabbed layout.

7.2 Project Structure

The project is organized into several key modules, each responsible for a specific stage of the data refinement process:

Input & Parsing Module: Uses json5 to ingest raw text or unformatted JSON strings. It acts as the gateway, cleaning initial syntax errors before further processing.

Refinement & Type Engine: The heart of the system that performs Automatic Type Inference. It scans every value to determine if it should be cast as an Integer, Float, Boolean, or Null, ensuring the data is not just "strings of text."

Naming Convention Module: A specialized module that uses Regular Expressions to identify word boundaries in keys and transform them into one of the four supported standards: snake_case, camelCase, kebab-case, or PascalCase.

Transformation Module: Handles structural changes. The Flattening logic converts a hierarchical tree into a flat key-value list (dot-notation), while the Unflattening logic uses a recursive builder to reconstruct the tree structure.

Validation & Quality Control Module: This module integrates jsonschema to validate the final output against a user-defined or default schema. It catches missing required fields and incorrect data structures, providing real-time error reporting.

Persistence & Export Module: Manages the History Log, tracking every refinement step taken during a session. It also handles the generation of the final .json file for user download.

7.3 User Interface (UI) Design

The UI is designed with a 7-Tab Layout for logical separation of tasks:

Core Refining: Main dashboard for type and case conversion.

JSON Schema: For structural validation.

Flatten/Unflatten: For hierarchy manipulation.

Merge JSON: For combining data sources.

History & Stats: For tracking and data insights.

Guide & Settings: For user assistance and configuration.

PROJECT WORKFLOW

5. PROJECT WORKFLOW

The development and execution of the JSON Refiner project followed a systematic lifecycle, moving from environment preparation to complex algorithmic implementation and final user-interface integration.

Step 1: Environment Setup & Requirement Analysis The initial phase involved setting up the Python environment and installing necessary dependencies such as gradio for the UI, json5 for flexible parsing, and jsonschema for data validation. This stage also defined the core requirements: automatic type inference, case conversion, and structural transformation.

Step 2: Designing the Core Refining Engine The "brain" of the project was developed in this stage. A centralized processing class was created to handle the primary logic. This includes the Automatic Type Inference algorithm, which scans raw text to identify if a value should be a Boolean, Integer, Float, or String, ensuring data integrity from the start.

Step 3: Implementing Case Conversion Standards To ensure professional-grade output, logic was implemented to standardize key names. Using Regular Expressions (Regex), the workflow integrated four major naming standards: snake_case, camelCase, kebab-case, and PascalCase. This allows users to normalize inconsistent data into a single preferred style.

Step 4: Developing Advanced Transformation Modules In this stage, the project workflow focused on complex structural changes. Algorithms for Flattening (converting nested JSON into dot-notation) and Unflattening (reconstructing objects from flat keys) were built. Additionally, a Merge function was developed to consolidate multiple JSON objects into one.

Step 5: Integration of Validation and History Tracking A robust validation layer was added using the JSON Schema (Draft 7) standard. The workflow also included a history-tracking mechanism that logs every transformation, providing users with a "Conversion Log" and statistical data (e.g., total keys processed, nesting depth).

Step 6: User Interface (UI) Construction Using the Gradio framework, the backend logic was connected to a front-facing interactive dashboard. The interface was organized into seven distinct functional tabs, ensuring that each feature (Refining, Schema, Flatten, etc.) is accessible and user-friendly.

Step 7: Testing, Debugging, and Deployment The final workflow step involved rigorous testing with "messy" real-world data samples. The system was checked for syntax error handling, validation accuracy, and interface responsiveness. Once verified, the project was deployed via Google Colab for cloud-based accessibility.

ALGORITHM

6. ALGORITHM

Experimental Analysis The implementation of the JSON Refiner Advanced Edition project was carried out through several structured stages, including environment setup, development of the core refining engine, interface creation, and comprehensive data testing. These steps ensure that the system accurately identifies data types and generates production-ready JSON based on the user's input.

Step 1: Setting Up the Development Environment The first step involves preparing the development environment by installing the required Python libraries. The Gradio framework is used for the interface, while JSON5 and JSONSchema are essential for handling flexible and valid data structures.

Command:

Bash

pip install gradio json5 jsonschema pydantic

Step 2: Importing Required Libraries After installing the libraries, necessary Python modules such as `json`, `re` (Regular Expressions), `gradio`, and `jsonschema` are imported. These libraries are crucial for parsing unstructured text, performing case conversions, and managing the multi-tab interface.

Step 3: Implementing the Core Refining Engine A sophisticated refining system is developed to handle automatic type inference. This engine analyzes input strings to distinguish between integers, floats, booleans, and null values. It also includes logic for naming convention transformations like **snake_case** and **camelCase**.

Step 4: Developing the User Interface The application interface is created using the **Gradio** framework with a tabbed layout. It provides seven functional tabs, including "Core Refining," "Schema Validation," and "Merge JSON." Each tab is designed with specific input areas and highlighted code output blocks for better user experience.

Step 5: Developing Transformation Logic (Flatten/Unflatten) Specific functions are implemented to handle complex nested JSON structures. The "Flatten" logic converts hierarchical data into a flat dot-notation format, while the "Unflatten" logic reverses this process, allowing for flexible data manipulation.

Step 6: Running and Testing the Application Finally, the application is executed and tested using various "messy" data samples. The system is verified for its ability to handle syntax errors, validate against schemas, and maintain a history log of all transformations. This ensures that the final JSON output is optimized for professional software development.

IMPLEMENTATION

6.IMPLEMENTATION

5..1 CODING:

Cell 1 – Install Transformers

```
!pip install -U transformers
```

CELL 2 — Import Pipeline and Load Model

```
from transformers import pipeline
pipe = pipeline("text-generation", model="ibm-granite/granite-3.3-2b-instruct")
messages = [
{"role": "user", "content": "Who are you?"},
]
pipe(messages)
```

Cell 3 – Load Model Directly

```
from transformers import AutoTokenizer, AutoModelForCausalLM
tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
messages = [
{"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
messages,
add_generation_prompt=True,
tokenize=True,
return_dict=True,
return_tensors="pt",
```

```
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)

print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))
```

Cell 4 – Install Required Libraries

```
!pip install -q --upgrade gradio json5 jsonschema python-dotenv pydantic
```

CELL 5 — Import Required Modules

```
import json
import json5
import pandas as pd
import gradio as gr
import re
from enum import Enum
from jsonschema import validate, ValidationError
```

Cell 6 – Case Style Enum

```
class CaseStyle(Enum):

SNAKE_CASE = "snake_case"
CAMEL_CASE = "camelCase"
KEBAB_CASE = "kebab-case"
PASCAL_CASE = "PascalCase"
```

Cell 7 – JSON Refiner Core Class

```
class JSONRefinerCore:

@staticmethod

def to_snake_case(text: str) -> str:
text = re.sub(r'[\s\-]+', '_', text)
text = re.sub(r'([a-z])([A-Z])', r'\1_\2', text)
return text.lower()

@staticmethod

def to_camel_case(text: str) -> str:
parts = text.split('_')
return parts[0] + ''.join(word.capitalize() for word in parts[1:])

@staticmethod

def to_kebab_case(text: str) -> str:
return text.replace('_', '-')
```

```
@staticmethod
def to_pascal_case(text: str) -> str:
return ''.join(word.capitalize() for word in text.split('_'))
```

Cell 8 – Convert Key Value Text to JSON

```
def parse_key_value_text(text):
result = {}
lines = text.split('\n')
for line in lines:
if ":" in line:
key, value = line.split(":", 1)
result[key.strip()] = value.strip()
return result
```

Cell 9 – Main JSON Refining Function

```
def refine_json(text_input):
if not text_input.strip():
return ""
data = parse_key_value_text(text_input)
json_output = json.dumps(data, indent=2)
return json_output
```

Cell 10 – Gradio Interface

```
with gr.Blocks() as demo:
gr.Markdown("# JSON Refiner Tool")
text_input = gr.Textbox(label="Input Key Value Text", lines=10)
output = gr.Code(language="json")
btn = gr.Button("Convert")
```

```
btn.click(refine_json, inputs=text_input, outputs=output)
demo.launch()
```

Cell 11 – Example Input

name: jarin

age: 20

email: jarin@gmail.com

Cell 12 – Import JSON Validation Libraries

```
import json5
from jsonschema import validate, ValidationError
```

Cell 13 – Normalize JSON Key

```
def normalize_key(key, case_style="snake_case"):
key = key.strip()
if case_style == "snake_case":
key = key.lower().replace(" ", "_")
elif case_style == "camelCase":
parts = key.split(" ")
key = parts[0].lower() + ".join(word.capitalize() for word in parts[1:])
return key
```

Cell 14 – Infer Data Type Automatically

```
def infer_type(value):
value = value.strip()
if value.lower() in ["true", "false"]:
return value.lower() == "true"
if value.isdigit():
```

```
return int(value)
try:
return float(value)
except:
return value
```

Cell 15 – Convert Text to Dictionary

```
def text_to_dict(text):
result = {}
lines = text.split("\n")
for line in lines:
if ":" in line:
key, value = line.split(":",1)
key = normalize_key(key)
value = infer_type(value)
result[key] = value
return result
```

Cell 16 – JSON Statistics Function

```
def json_stats(data):
return {
"total_keys": len(data),
"value_types": list(set(type(v).__name__ for v in data.values()))
}
```

Cell 17 – Remove Null Values

```
def remove_null_values(data):
```

```
return {k:v for k,v in data.items() if v is not None}
```

Cell 18 – Flatten JSON

```
def flatten_json(data, parent_key="", sep="."):
items = []
for k, v in data.items():
new_key = parent_key + sep + k if parent_key else k
if isinstance(v, dict):
items.extend(flatten_json(v, new_key, sep).items())
else:
items.append((new_key, v))
return dict(items)
```

Cell 19 – Merge Multiple JSON Objects

```
def merge_json(obj1, obj2):
result = obj1.copy()
result.update(obj2)
return result
```

Cell 20 – Validate JSON

```
def validate_json(json_text):
try:
json.loads(json_text)
return "Valid JSON"
except json.JSONDecodeError as e:
return str(e)
```

Cell 21 – Case Conversion Function

```
def convert_case(data, case_style):
```

```
new_data = {}
for key, value in data.items():
new_key = normalize_key(key, case_style)
new_data[new_key] = value
return new_data
```

Cell 22 – History Tracking

```
transformation_history = []
def save_history(input_text, output_json):
transformation_history.append({
        "time": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "input": input_text,
        "output": output_json
    })
```

Cell 23 – Download History

```
def download_history():
report = ""
for item in transformation_history:
report += f"Time: {item['time']}\n"
report += f"Input: {item['input']}\n"
report += f"Output: {item['output']}\n\n"
return report
```

Cell 24 – Gradio Input Section

```
text_input = gr.Textbox(
```

```
label="Enter Key Value Text",
lines=10
)
```

Cell 25 – Gradio Output Section

```
json_output = gr.Code(
language="json",
label="JSON Output"
)
```

Cell 26 – Button Action

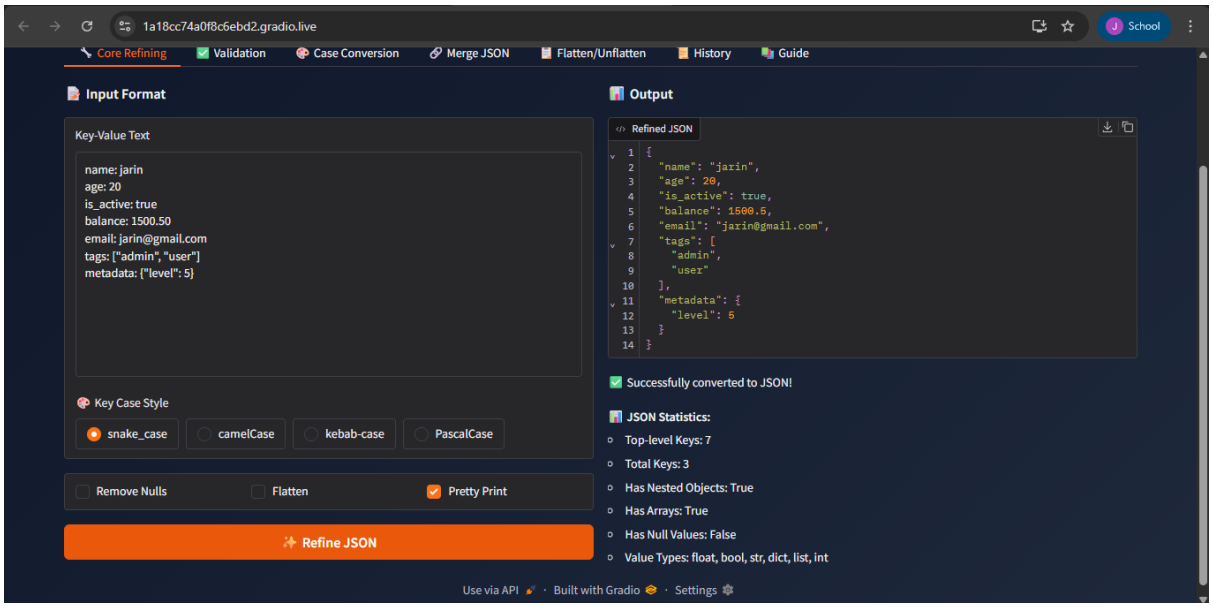
```
convert_btn = gr.Button("Convert to JSON")
convert_btn.click(
refine_json,
inputs=text_input,
outputs=json_output
)
```

Cell 27 – Launch Application

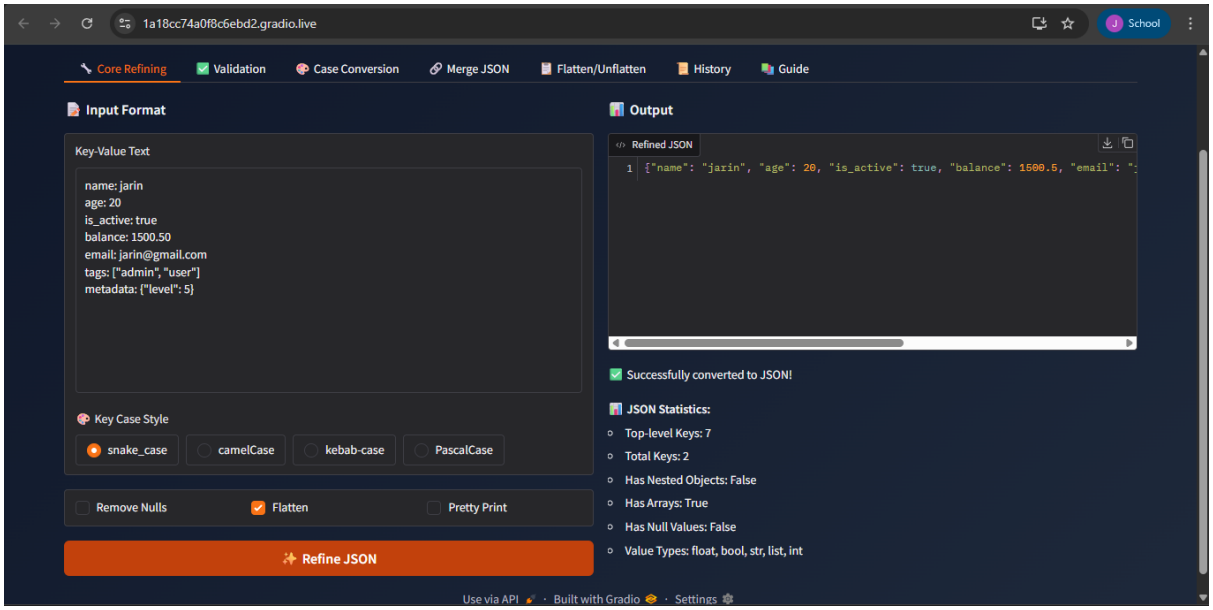
```
demo.launch(share=True)
```

5.2 OUTPUT:

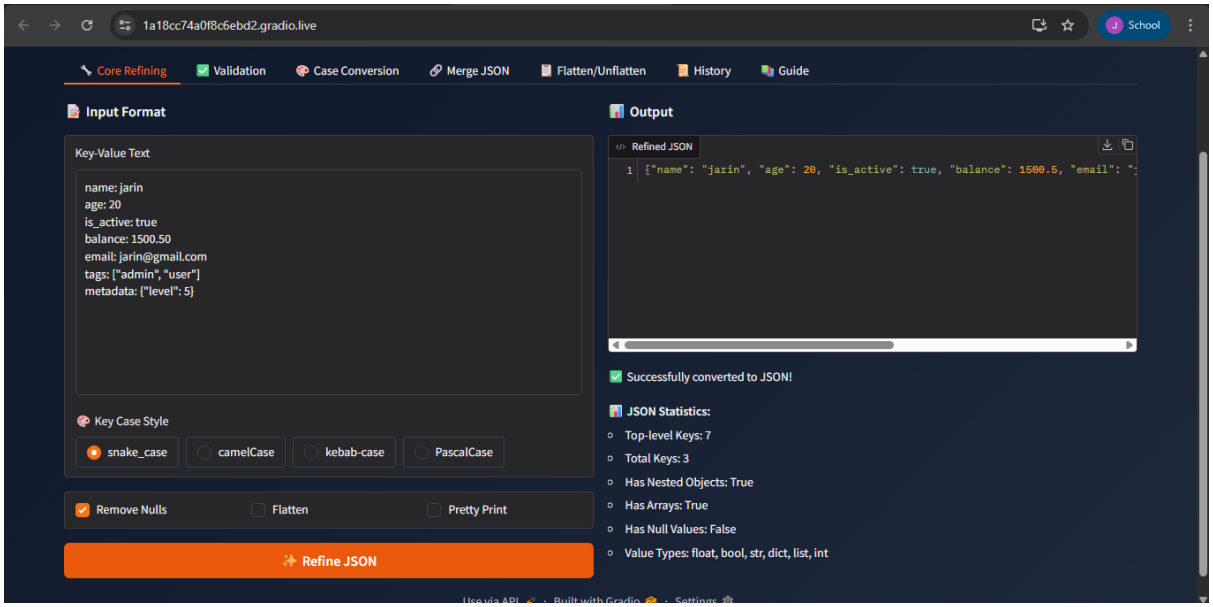
1.Core Refining a. Text to JSON Conversion (Pretty print)



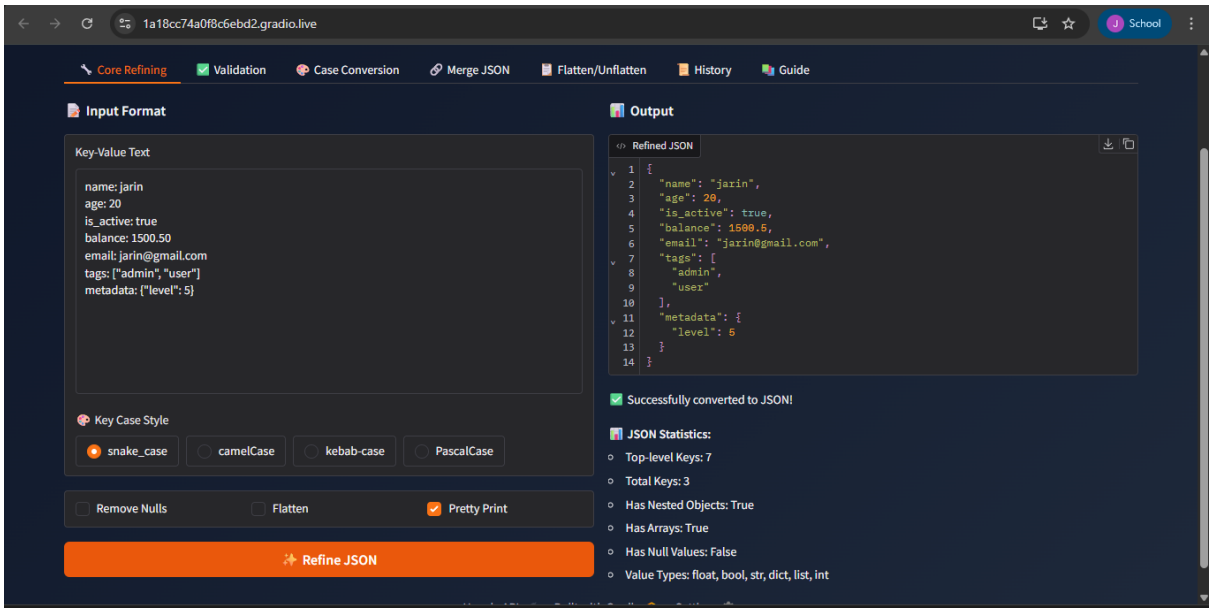
Core Refining a. Text to JSON Conversion (Flattern)



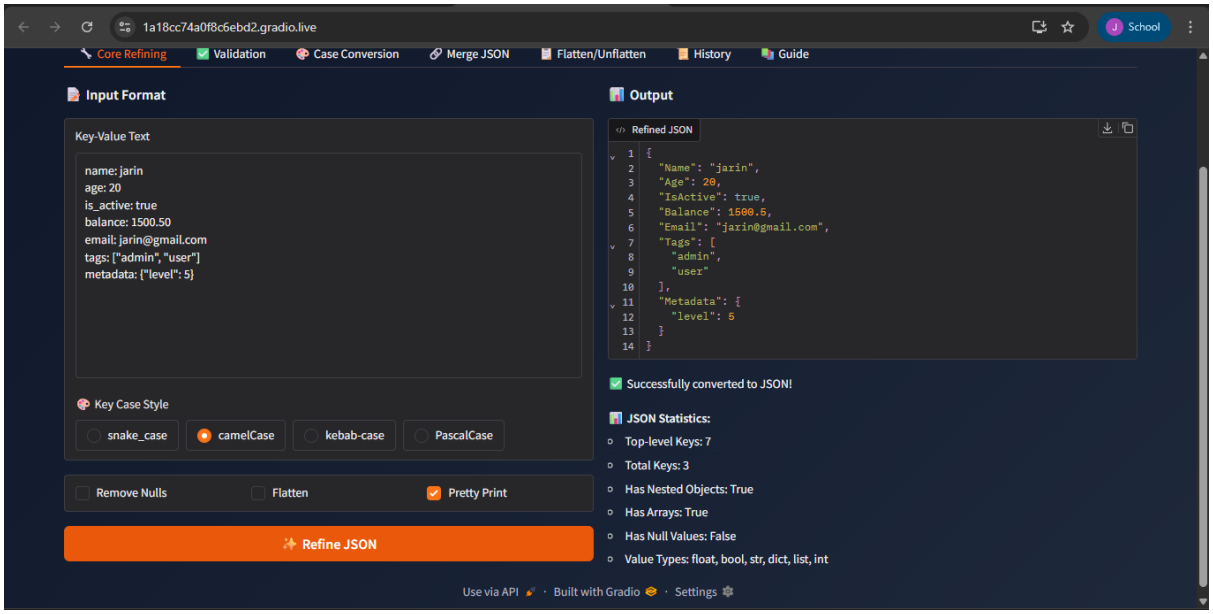
Core Refining a. Text to JSON Conversion (Remove Nulls)



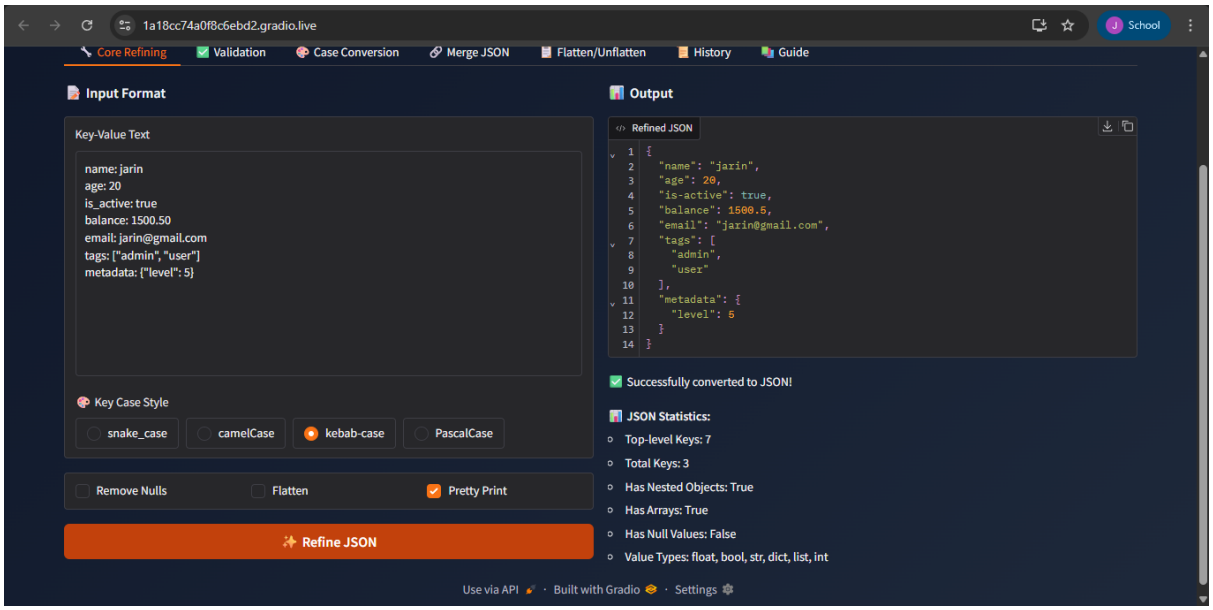
Core Refining (Snake case)



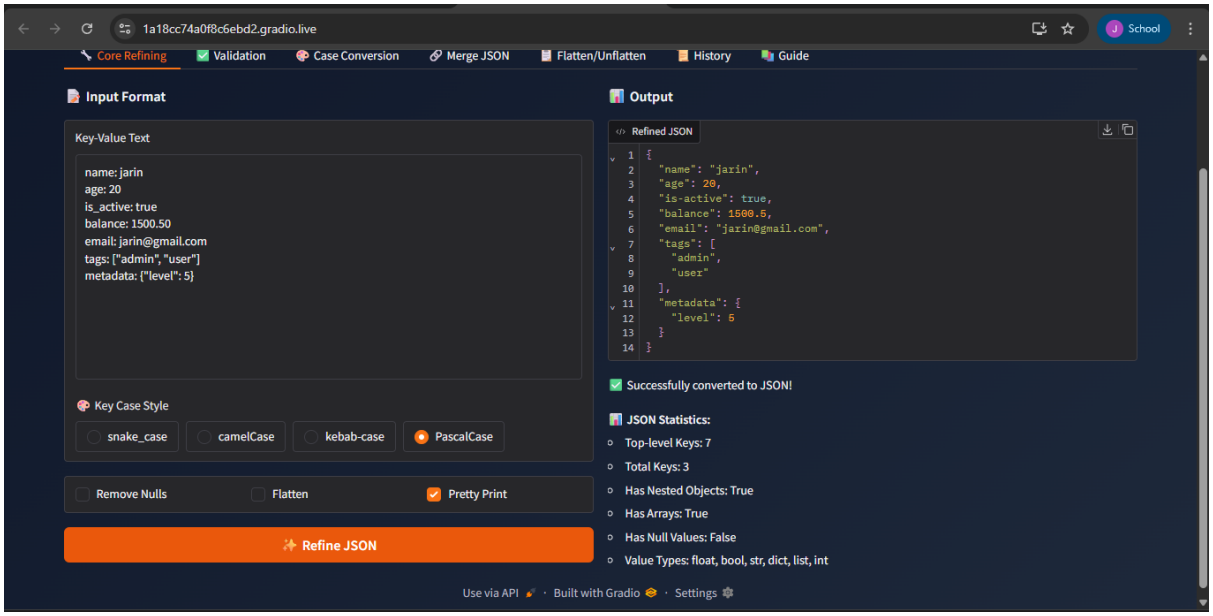
Core Refining (Camel Case)



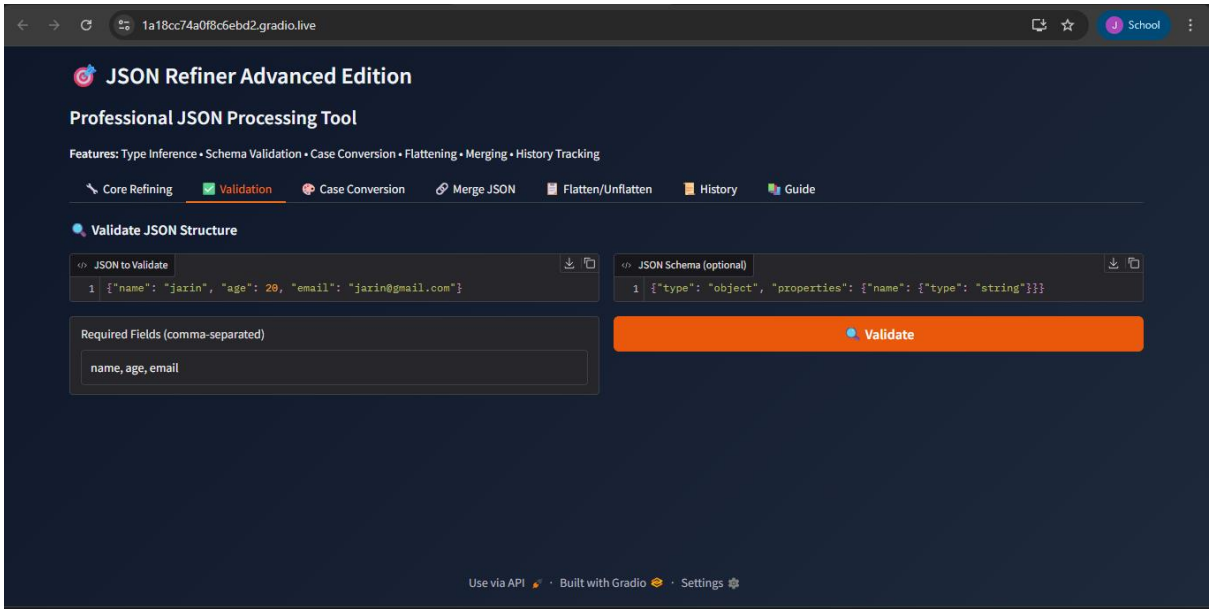
Core Refining (Kebab Case)



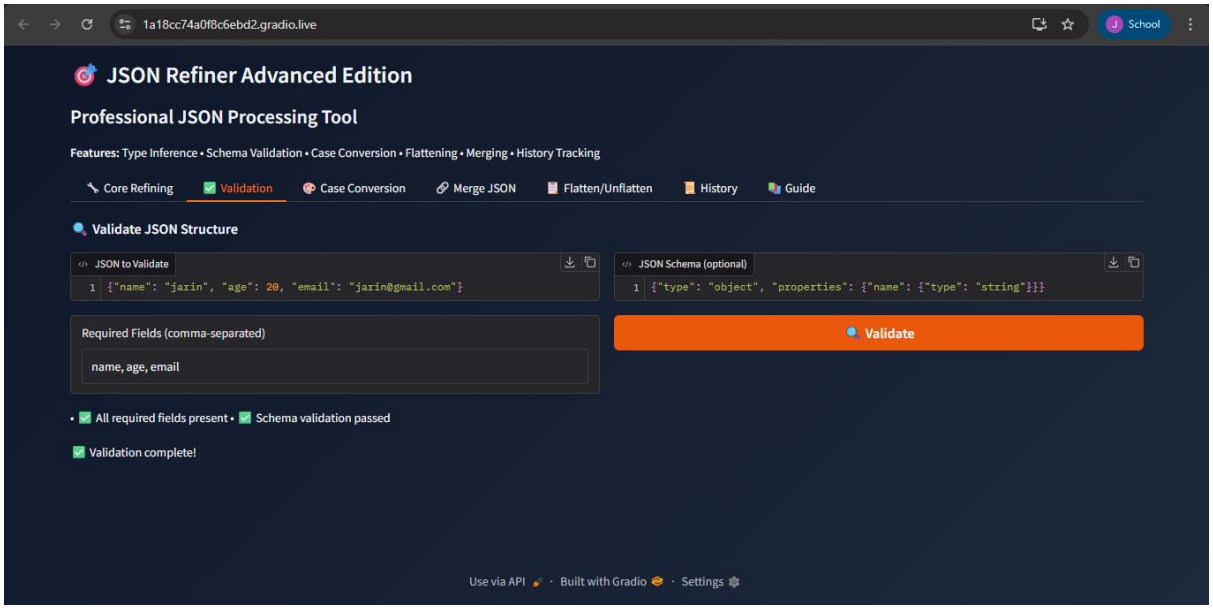
Core Refining (Pascal Case)



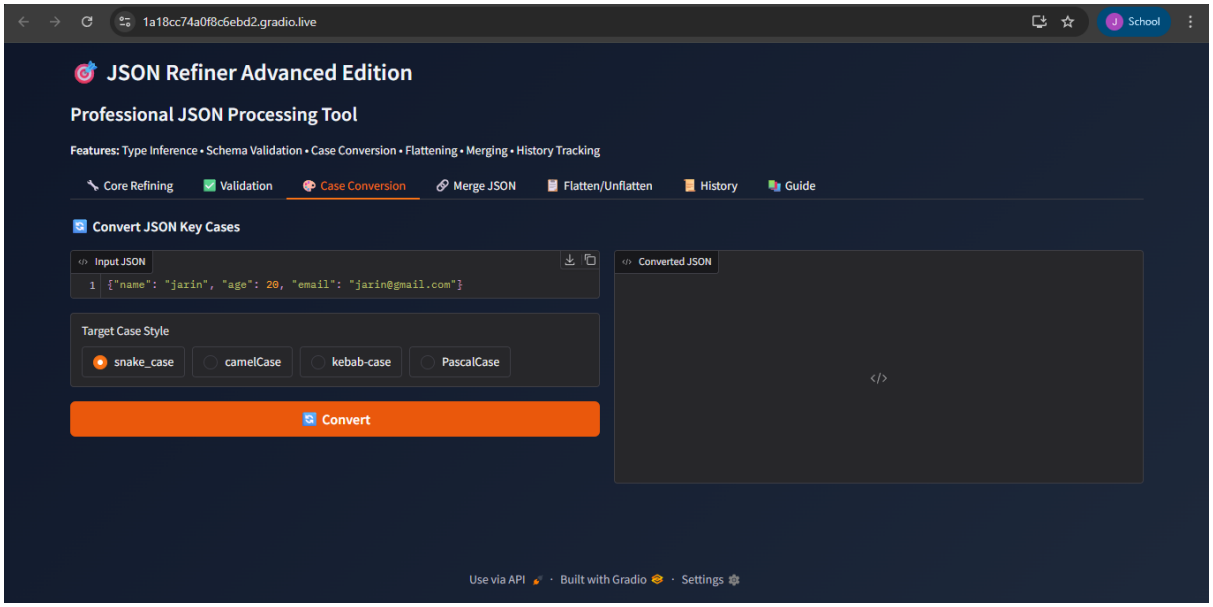
2.Validation



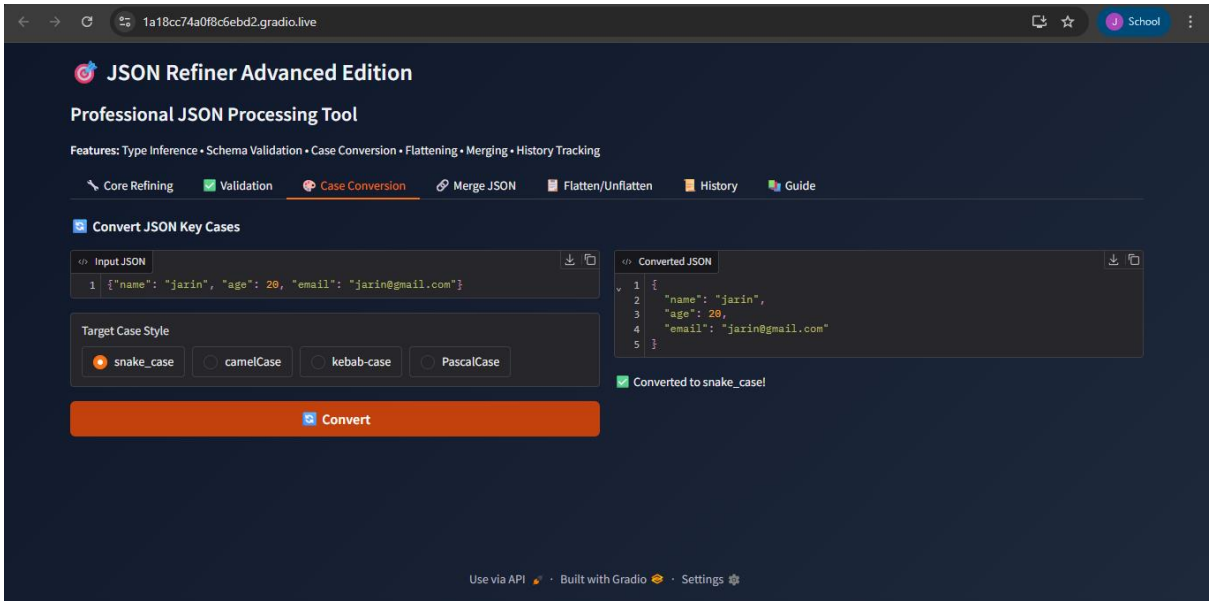
Validation Complete (Schema Validation)



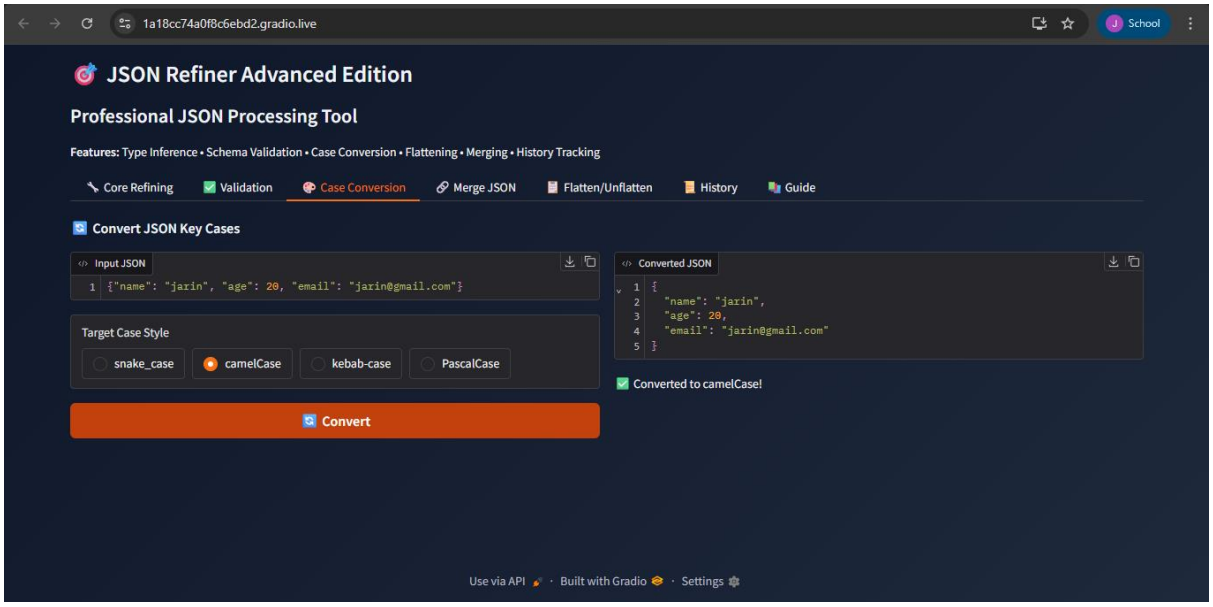
3.Case Conversion



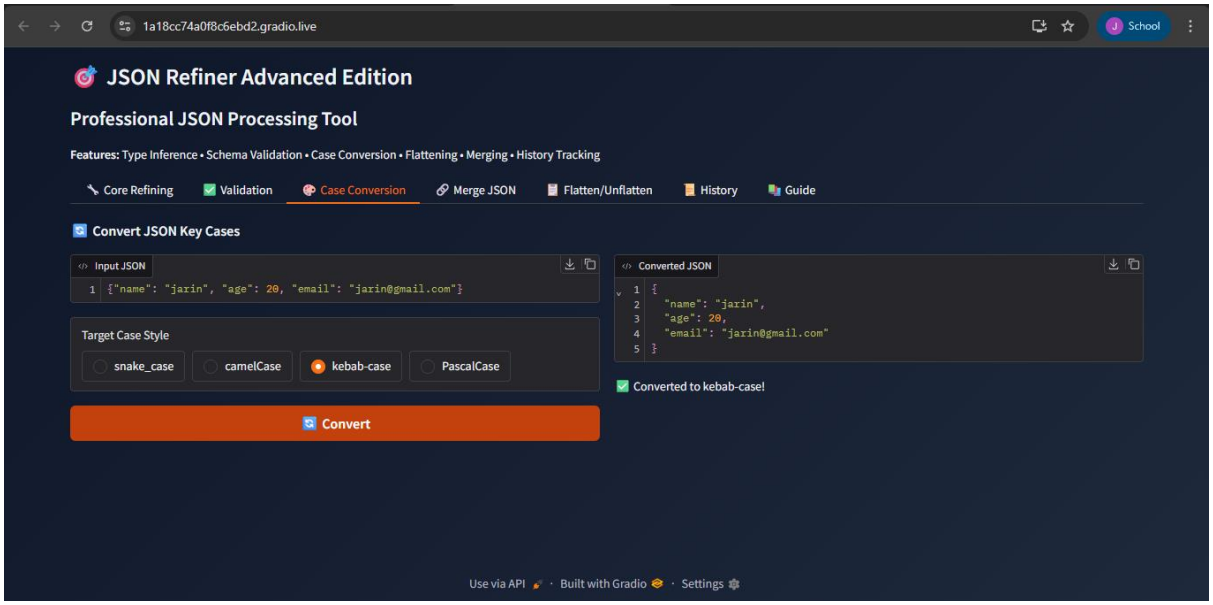
Case Conversion (Snake case)



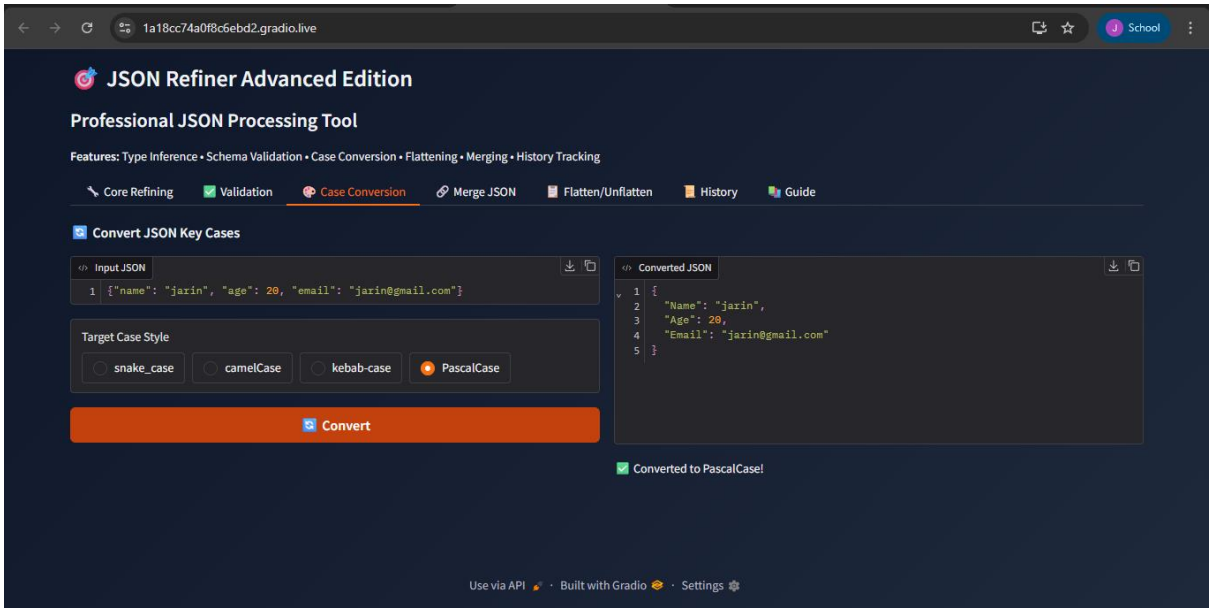
Case Conversion (Camel case)



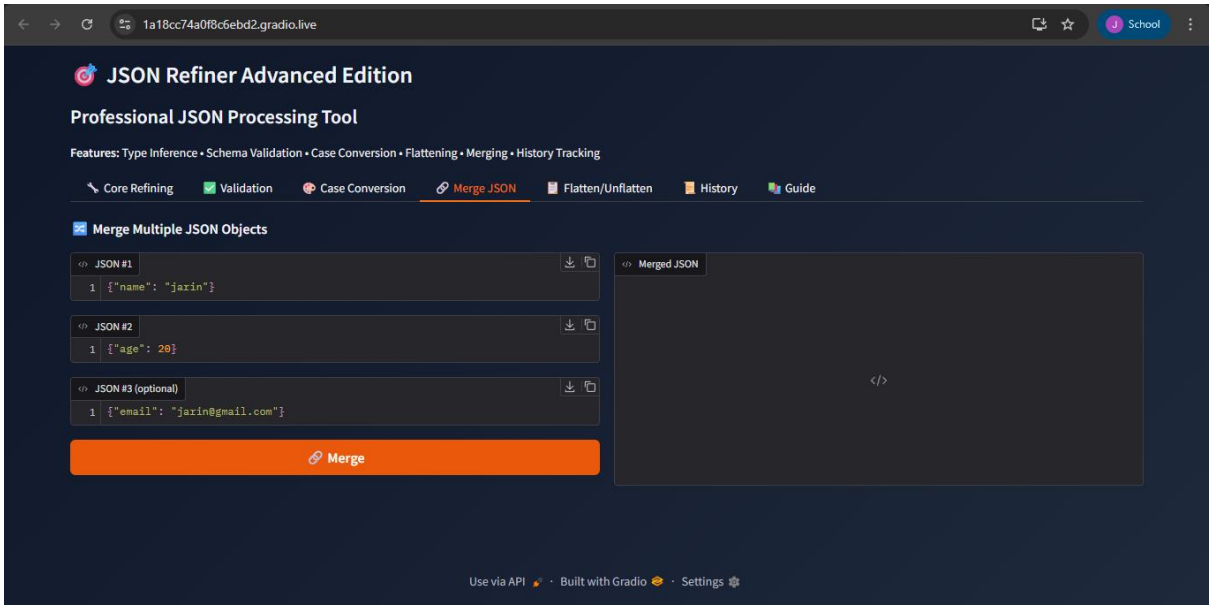
Case Conversion (kebab case)



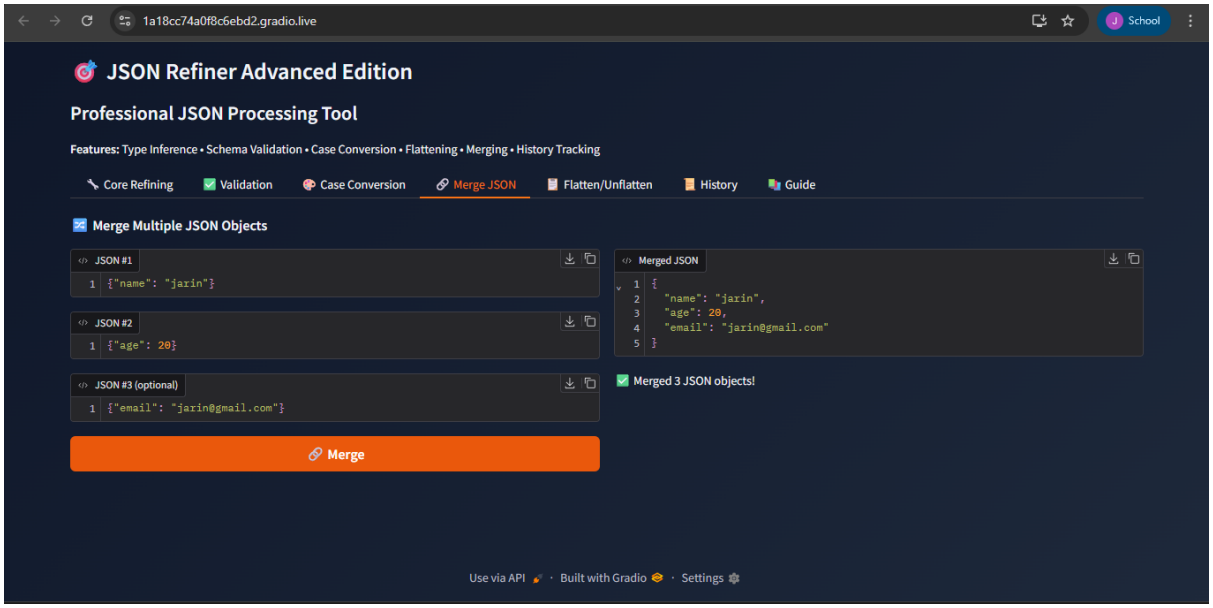
Case Conversion (Pascal case)



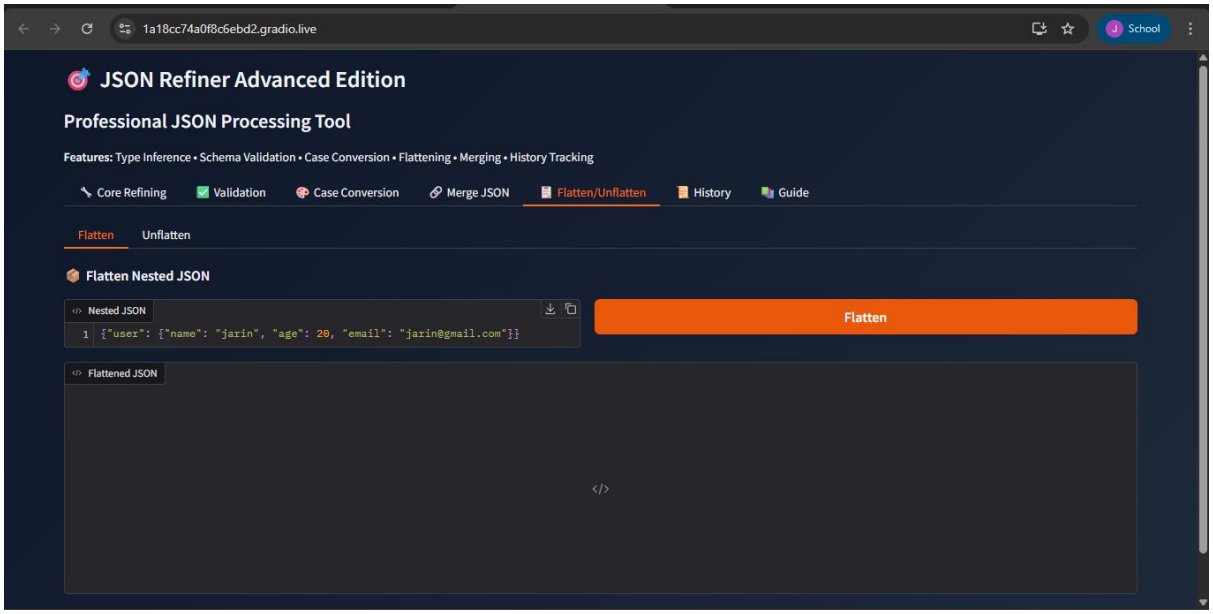
4.Merge JSON



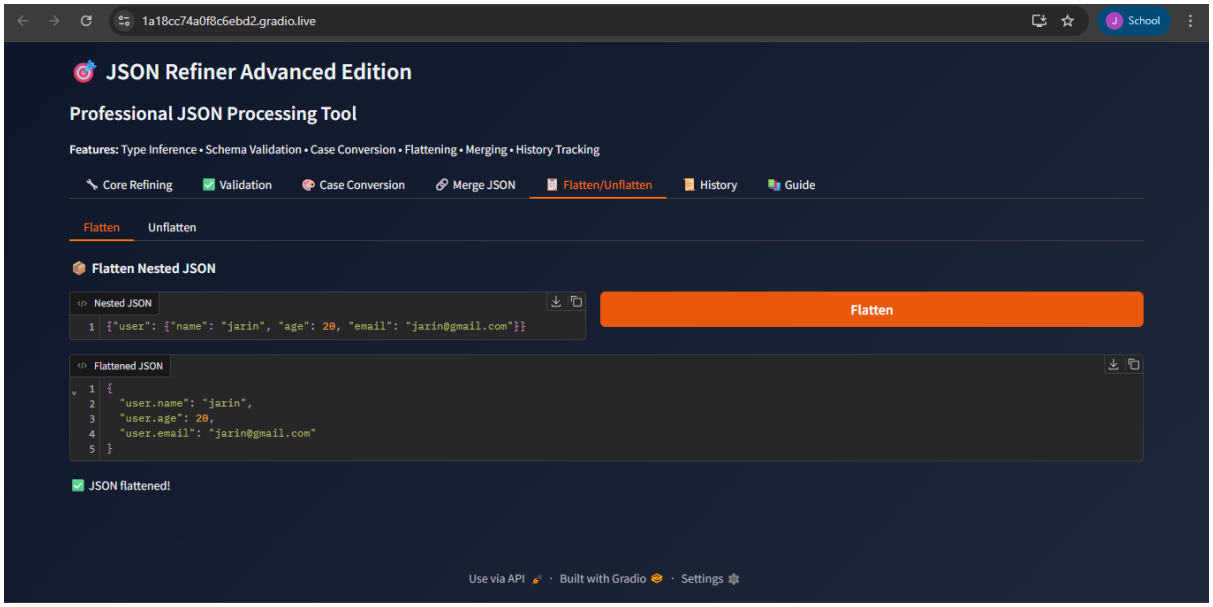
Merge Multiple JSON Data



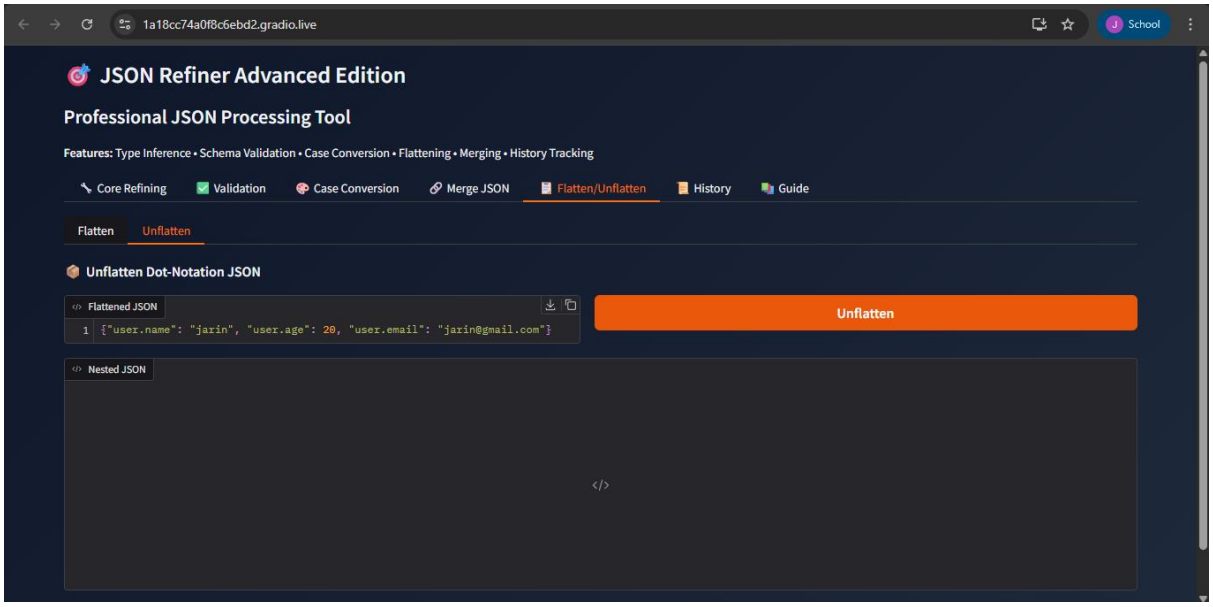
5.Flattan



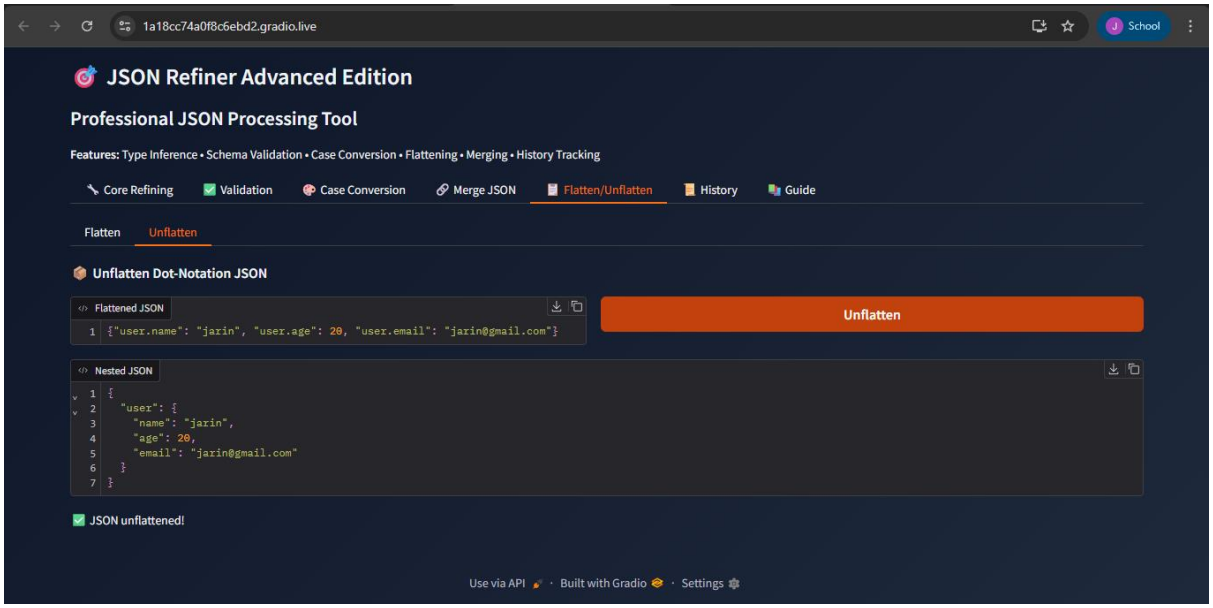
JSON Flattened



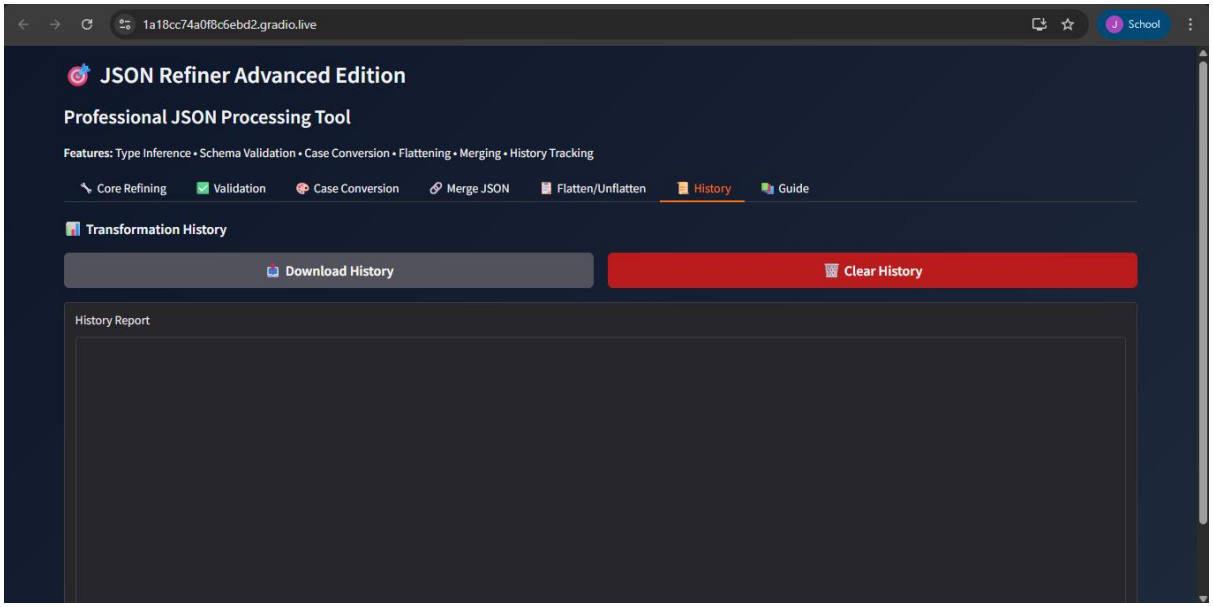
Unflatten



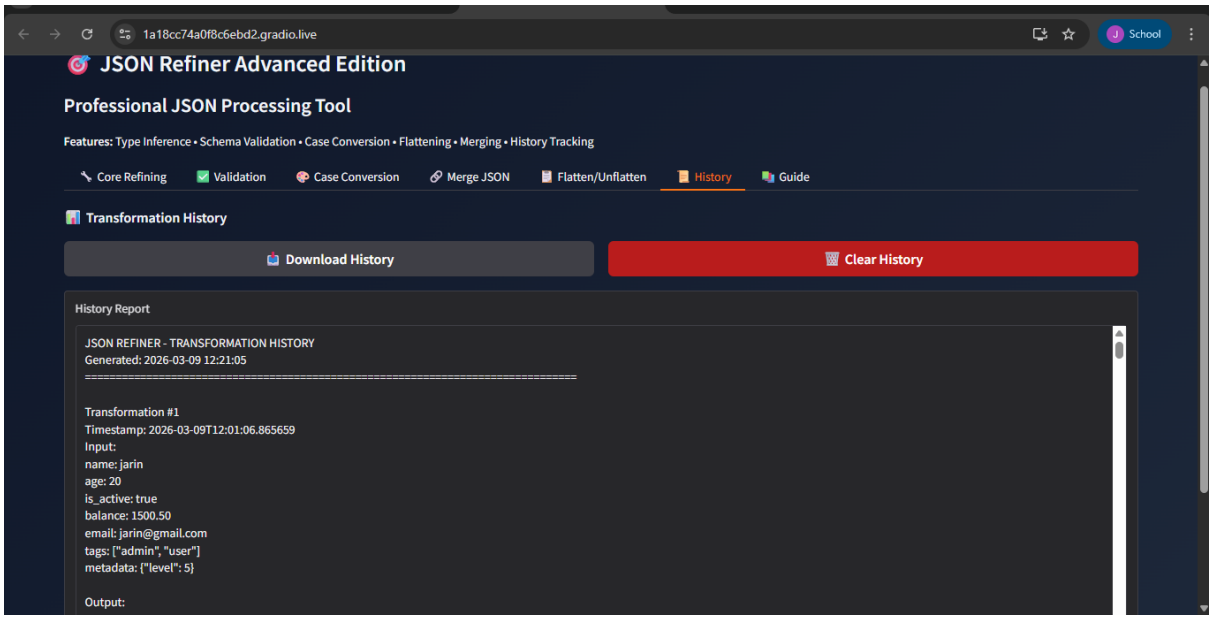
JSON Unflatten



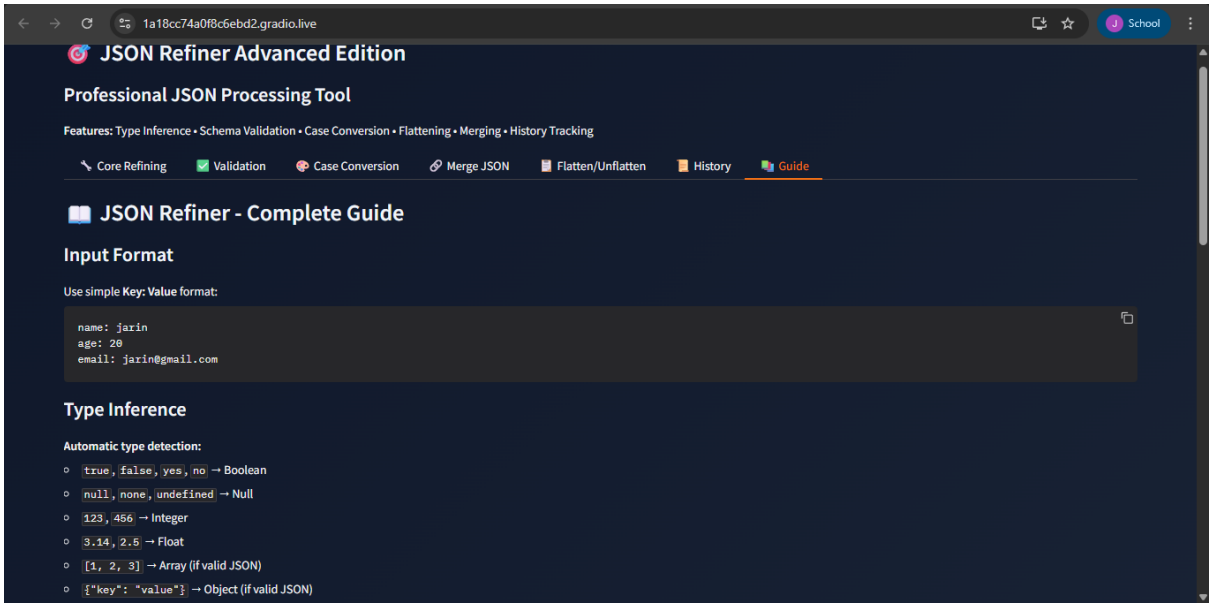
6.History



Download History



7.Use Guide


1a18cc74a0f8c6ebd2.gradio.live
School
JSON Refiner Advanced Edition
Professional JSON Processing Tool
Features: Type Inference • Schema Validation • Case Conversion • Flattening • Merging • History Tracking
Core Refining
Validation
Case Conversion
Merge JSON
Flatten/Unflatten
History
Guide
JSON Refiner - Complete Guide
Input Format
Use simple Key: Value format:
name: jarin
age: 20
email: jarin@gmail.com
Type Inference
Automatic type detection:
true, false, yes, no → Boolean
null, none, undefined → Null
123, 456 → Integer
3.14, 2.5 → Float
[1, 2, 3] → Array (if valid JSON)
{"key": "value"} → Object (if valid JSON)

CONCLUSION

6.CONCLUSION

The **JSON Refiner Advanced Edition** project was developed to simplify the process of creating professional, structured, and validated JSON data without requiring manual formatting or advanced scripting knowledge. In traditional data management, developers and analysts must manually clean, type-check, and format JSON structures, which is both time-consuming and prone to human error. This project addresses those challenges by providing an intelligent, automated system that refines "messy" data into production-ready JSON based on simple user inputs.

The application allows users to enter raw key-value pairs or unformatted JSON strings and instantly apply transformations. With features like **Automatic Type Inference**, the system intelligently distinguishes between data types such as integers, booleans, and strings. Users can also standardize their data using industry-standard naming conventions like **camelCase** and **snake_case**. This makes data preparation faster, more accurate, and accessible for users who may not be experts in JSON syntax.

Another significant achievement of this project is the highly interactive and user-friendly interface developed using the **Gradio** framework. The interface is organized into seven functional tabs, allowing users to handle complex tasks like **Schema Validation**, **Flattening/Unflattening**, and **Merging** multiple data sources in one place. The application provides real-time feedback through color-coded status messages and detailed statistics, helping users understand the structure and quality of their data immediately.

The system also includes advanced utility features such as a **History Log** for tracking changes and a download option for exporting the final refined JSON file. These features make the tool highly practical for developers, data scientists, and students who need to prototype APIs or clean datasets quickly.

In conclusion, the **JSON Refiner Advanced Edition** demonstrates how automation and intelligent parsing algorithms can significantly improve the data refinement process. By simplifying complex structural transformations and providing built-in validation, the system helps users transform raw, inconsistent information into high-quality, functional data structures

efficiently. This project serves as a comprehensive tool for anyone looking to maintain high data standards in the modern digital era.